

Building a Robust Part-of-Speech Tagger Using a Hidden Markov Model

Probabilistic Graphical Models · Class I4-AMS-DAS-A (2025–2026) ·

Team 3

HEAM VISAL — e20221349 ·

MON Sreylin — e20221701 ·

CHOUB Botumraksa — e20221709 ·

CHIV Mengchou — e20221028

Lecturer: Mr. PHOK Ponna

Outline

This presentation walks through our end-to-end HMM-based POS tagger, from problem definition through implementation and evaluation.

01	02
Problem and Objective	Dataset
POS tagging task, ambiguity, and project goals	NLTK Brown Corpus, tagset, and train/test split
03	04
HMM Approach	Parameter Estimation
Model structure, hidden states, and joint probability	MLE, Laplace smoothing, and OOV handling
05	06
Viterbi Decoding	Results and Error Analysis
Log-space dynamic programming algorithm	93.8% accuracy, confusion patterns
07	
Conclusion and DEMO	

1. POS Tagging: Problem and Objective

Part-of-speech tagging assigns a grammatical label, such as NOUN, VERB, or ADJ, to every word. The correct label can change depending on context.

Lexical Ambiguity in Action

"My watch is broken."

→ *watch* = NOUN (a timepiece)

"Watch the dog."

→ *watch* = VERB (to observe)

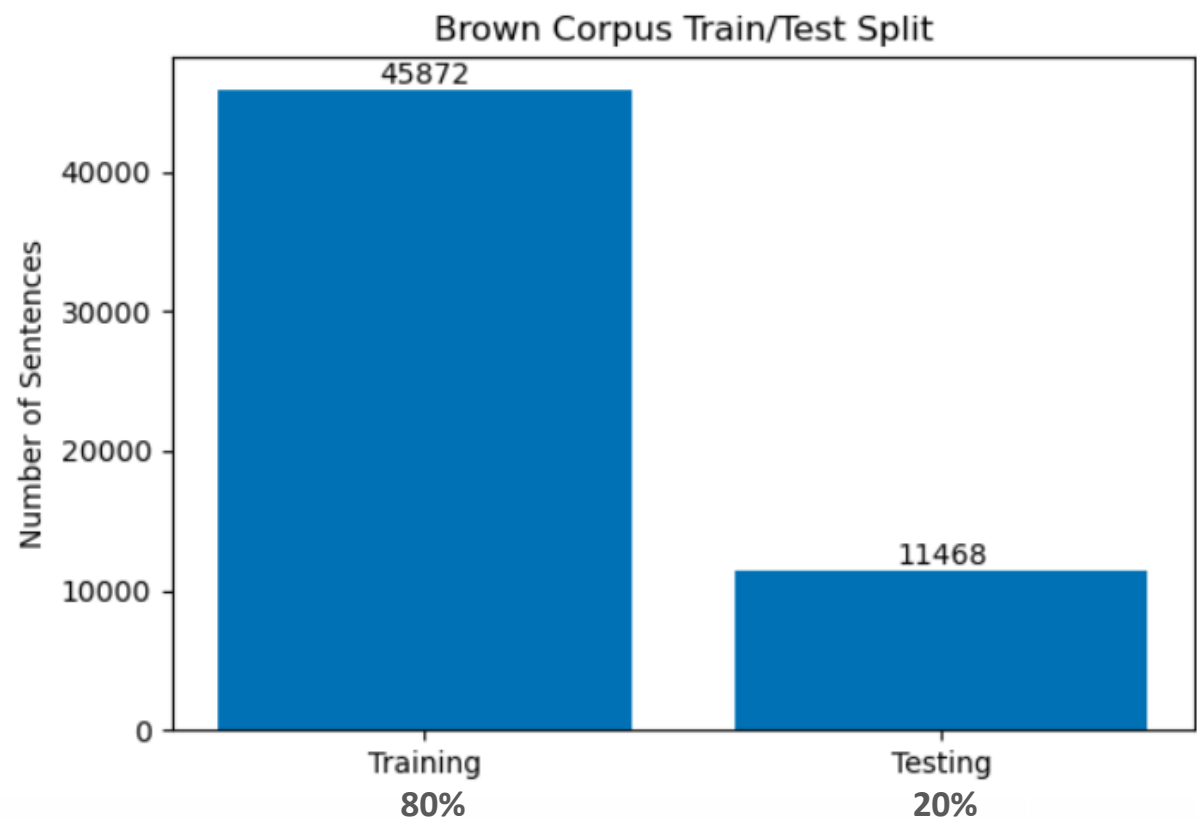
The same word can require different tags in different contexts.

Project Objectives

- Build an HMM POS tagger from scratch using the Brown Corpus
- Handle unseen words using Laplace smoothing and <OOV>
- Prevent numerical underflow using log-space arithmetic
- Evaluate word-level accuracy on the held-out 20% test set

2. Dataset and Preprocessing

We used 57,340 tagged sentences from the **NLTK Brown Corpus** with 12 Universal POS tags. Sentences were shuffled using seed 42, normalized to lowercase, and split into 80% training and 20% testing data.



Dataset at a Glance

57,340

Total sentences

12

Universal POS tags

45,153

Training vocabulary

Seed: 42

Reproducible split

The split was performed at the sentence level. Test sentences never contributed counts during training, preventing data leakage.

3. Hidden Markov Model Approach

Model Structure

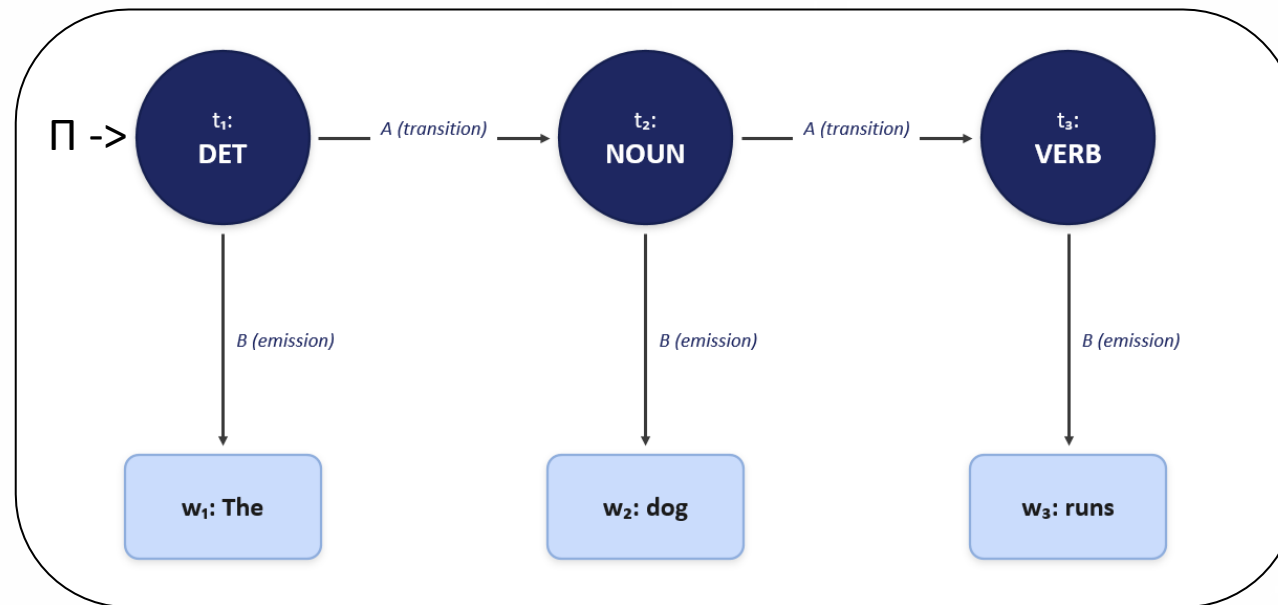
- **Hidden states:** POS tags $t_1, \dots, t_n$
- **Observations:** words $w_1, \dots, w_n$
- **Transition A:** probability of the next tag
- **Emission B:** probability of a word given its tag
- **Initial π :** probability of the first tag

Joint Probability

$$P(t_1, \dots, t_n, w_1, \dots, w_n) = \pi(t_1) B_{t_1}(w_1) \prod_{i=2}^n A_{t_{i-1}, t_i} B_{t_i}(w_i)$$

Where π = initial probability, **A** = tag transition, **B** = word emission.

Graphical Chain



The chain illustrates first-order transitions between hidden tag states and local emissions of observed words.

4. Parameter Estimation and OOV Handling

The initial, transition, and emission probabilities are estimated using MLE. Laplace smoothing with $\alpha = 1.0$ is applied only to emission probabilities.

Smoothed Emission Probability

$$B(t, w) = \frac{\text{Count}(w \text{ emitted by } t) + \alpha}{\text{Count}(t) + \alpha(|V| + 1)}$$

The $|V| + 1$ term accounts for all known vocabulary words plus one OOV outcome.

OOV probability

$$B(t, \text{OOV}) = \frac{\alpha}{\text{Count}(t) + \alpha(|V| + 1)}$$

Three Parameter Types

π – Initial Tag (Probability)

Count how often tag t begins a sentence

A – Transition Probability

$P(t_j | t_i)$ - Count how often tag j follows tag i

B – Emission Probability

$P(w | t)$ - Count how often word w appears with tag t

 Unseen words are mapped to <OOV>. Smoothing gives <OOV> a non-zero emission probability for every tag.

5. Log-Space Viterbi Decoder

Multiplying many small probabilities can underflow to zero. Log-space converts multiplication into addition while preserving the most likely tag sequence.

Log-Space Recurrence

$$\log v_t(j) = \max_i [\log v_{t-1}(i) + \log A(i, j)] + \log B(j, w_t)$$

A small value $\varepsilon = 10^{-12}$ is added before taking logarithms to avoid $\log(0)$.

Four-Step Decoding Process

1

Initialize

Set log-probabilities for position $t = 1$ using π and B
 $\log v_1(j) = \log \pi(j) + \log B(j, w_1)$

2

Forward Pass

Compute the best score for every tag at each position

3

Store Backpointers

Record the previous tag that produced each best score

4

Backtrack

Start from the best final tag and recover the full sequence

6. Results And Error Analysis

Results and Evaluation

The model achieved 93.8% word-level accuracy on 232,177 words from the held-out test set.

93.8%

Accuracy
Word-Level Accuracy

232,177

Test Words
Total words in the held-out test set

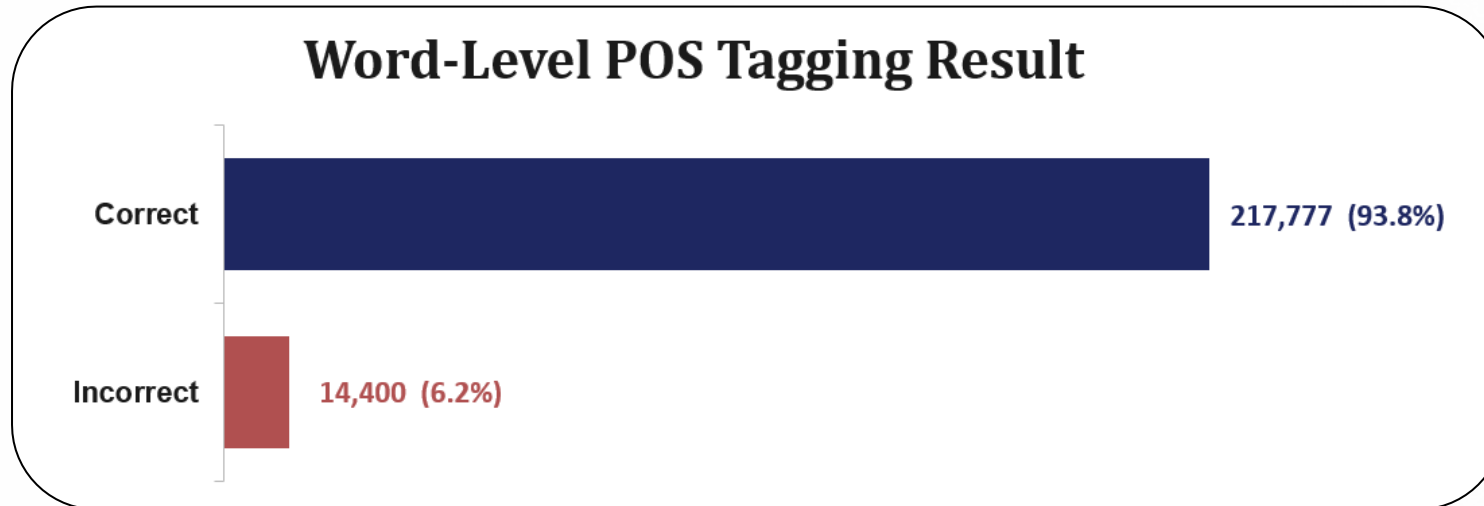
5,050

OOV Words
Unseen words (2.18% of test set)

14,400

Errors
Words tagged incorrectly by the model

Correct vs. Incorrect Tag Predictions



Error Analysis

Of 14,400 errors, most arise from **lexical ambiguity**, the **first-order Markov limitation**, and **rare or OOV words**.

Main Error Case

With tips, the girls **average** between \$150 and \$200 a week, depending on basic salary.

Illustrative Error

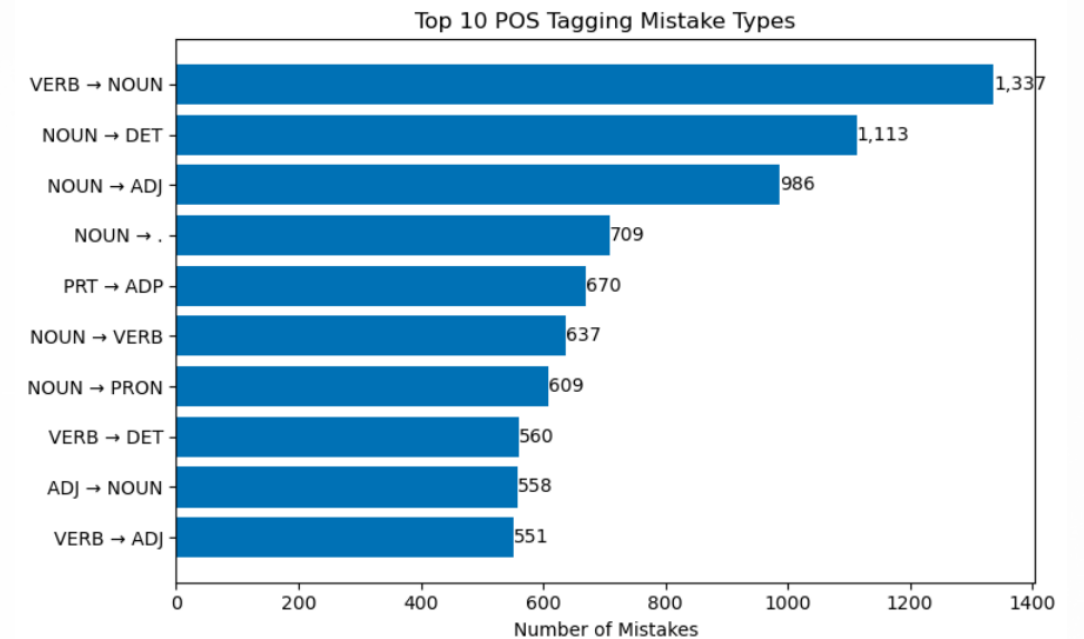
True tag: **VERB** · Predicted: **NOUN**

1. “Average” can be a NOUN, ADJ, or VERB.
2. Local transition and emission scores favored NOUN.
3. The first-order HMM only considers the previous tag, so it cannot use the full sentence meaning.

Main Causes

- **Lexical ambiguity:** same word, different POS across contexts
- **First-order limit:** only one previous tag considered
- **Rare / OOV words:** insufficient training evidence

Top Tag Confusion Pairs



Top 3 from the graph :

- VERB predicted as NOUN: 1,337
- NOUN predicted as DET: 1,113
- NOUN predicted as ADJ: 986

7. Conclusion And DEMO

93.8% Accuracy

Strong performance on 232,177 test words using a first-order HMM

OOV Handling

Laplace smoothing with $\alpha = 1.0$ allowed the model to decode 5,050 unseen words.

Numerical Stability

Log-space Viterbi prevented underflow across long sequences

Current Limitations

- Lexical ambiguity unresolved without richer features
- First-order Markov assumption limits long-range context
- Rare and OOV words remain challenging

Future Directions

- Add morphological features (prefixes, suffixes)
- Explore second-order or higher-order HMMs
- Compare against contextual neural models (e.g., BiLSTM-CRF)

DEMO